

DBMS WEEK 2 PROJECT REPORT

PRODUCT AND CATEGORY MANAGEMENT SYSTEM

INTRODUCTION

The Product and Category Management System is designed for an e-commerce company to maintain information about products and their categories.

Each product belongs to a particular category. The system stores important product details such as product name, price, and stock availability.

The project demonstrates the use of SQL tables, primary keys, foreign keys, constraints, data insertion, CRUD operations, and category-wise reports.

3. OBJECTIVES

The main objectives of this project are:

- To create Category and Product tables.
- To establish a relationship between the two tables.
- To use Primary Key and Foreign Key constraints.
- To apply NOT NULL, CHECK, and UNIQUE constraints.
- To insert product and category records.
- To perform CRUD operations.
- To generate category-wise reports.

TABLE STRUCTURE

4.1 Category Table

The Category table contains information about the different product categories.

SQL Query

```
CREATE TABLE Category (  
    Category_ID NUMBER PRIMARY KEY,  
    Category_Name VARCHAR2(50) NOT NULL UNIQUE  
);
```

```
SQL> CREATE TABLE Product (  
2     Product_ID NUMBER PRIMARY KEY,  
3     Product_Name VARCHAR2(100) NOT NULL,  
4     Price NUMBER(10,2) CHECK (Price > 0),  
5     Stock NUMBER CHECK (Stock >= 0),  
6     Category_ID NUMBER,  
7     CONSTRAINT fk_product_category  
8         FOREIGN KEY (Category_ID)  
9         REFERENCES Category(Category_ID)  
10 );
```

Table created.

```
SQL> DESC Product;
```

Name	Null?	Type
PRODUCT_ID	NOT NULL	NUMBER
PRODUCT_NAME	NOT NULL	VARCHAR2(100)
PRICE		NUMBER(10,2)
STOCK		NUMBER
CATEGORY_ID		NUMBER

Product Table

The Product table contains product information such as product ID, product name, price, stock, and category ID.

SQL Query

```
CREATE TABLE Product (  
    Product_ID NUMBER PRIMARY KEY,  
    Product_Name VARCHAR2(100) NOT NULL,  
    Price NUMBER(10,2) CHECK (Price > 0),  
    Stock NUMBER CHECK (Stock >= 0),  
    Category_ID NUMBER,  
    CONSTRAINT fk_product_category  
        FOREIGN KEY (Category_ID)  
        REFERENCES Category(Category_ID)  
);
```

```
SQL> DESC Product;  
Name                               Null?    Type  
-----  
PRODUCT_ID                        NOT NULL NUMBER  
PRODUCT_NAME                      NOT NULL VARCHAR2(100)  
PRICE                             NUMBER(10,2)  
STOCK                             NUMBER  
CATEGORY_ID                       NUMBER  
  
SQL> SELECT * FROM Category;
```

RELATIONSHIP BETWEEN TABLES

The relationship between Category and Product is **One-to-Many** (1 → *).

One category can contain multiple products.

Category_ID is the Primary Key in the Category table and acts as a Foreign Key in the Product table.

Relationship

Category

Category_ID → Primary Key

↓

Product

Category_ID → Foreign Key

Screenshot - Table Structure / Relationship

CATEGORY_ID	CATEGORY_NAME
1	Mobile Accessories
2	Footwear
3	Stationery
4	Kitchen Tools
5	Fitness Equipment

6. CONSTRAINTS USED

The following constraints are used in the project:

Primary Key

Uniquely identifies each record.

- Category_ID
- Product_ID

Foreign Key

Connects the Product table with the Category table.

- Product.Category_ID

NOT NULL

Ensures that important fields cannot contain empty values.

- Category_Name
- Product_Name

CHECK

Ensures that valid values are entered.

- Price must be greater than 0.
- Stock must be greater than or equal to 0.

UNIQUE

Prevents duplicate category names.

7. INSERTING CATEGORY DATA

SQL Query

```
INSERT INTO Category VALUES (1, 'Mobile Accessories');
INSERT INTO Category VALUES (2, 'Footwear');
INSERT INTO Category VALUES (3, 'Stationery');
INSERT INTO Category VALUES (4, 'Kitchen Tools');
INSERT INTO Category VALUES (5, 'Fitness Equipment');

COMMIT;
```

8. INSERTING PRODUCT DATA

SQL Query

```
INSERT INTO Product VALUES (201, 'Power Bank', 1799, 35, 1);
INSERT INTO Product VALUES (202, 'Wireless Charger', 1299, 28, 1);
INSERT INTO Product VALUES (203, 'USB-C Cable', 499, 60, 1);
INSERT INTO Product VALUES (204, 'Running Shoes', 2899, 22, 2);
INSERT INTO Product VALUES (205, 'Casual Sandals', 999, 31, 2);
INSERT INTO Product VALUES (206, 'Notebook Set', 349, 75, 3);
INSERT INTO Product VALUES (207, 'Gel Pen Pack', 199, 90, 3);
INSERT INTO Product VALUES (208, 'Electric Kettle', 2199, 16, 4);
INSERT INTO Product VALUES (209, 'Vegetable Chopper', 699, 27, 4);
INSERT INTO Product VALUES (210, 'Yoga Mat', 899, 24, 5);
INSERT INTO Product VALUES (211, 'Resistance Bands', 599, 40, 5);
```

```
COMMIT;
```

Screenshot - Product Data

PRODUCT_ID		
PRODUCT_NAME		
PRICE	STOCK	CATEGORY_ID
208		
Electric Kettle	2199	16
		4
209		
Vegetable Chopper	699	27
		4
PRODUCT_ID		
PRODUCT_NAME		
PRICE	STOCK	CATEGORY_ID
210		
Yoga Mat	899	24
		5
211		
Resistance Bands		
PRODUCT_ID		
PRODUCT_NAME		
PRICE	STOCK	CATEGORY_ID
599	40	5

PRODUCT_ID			
PRODUCT_NAME			
PRICE	STOCK	CATEGORY_ID	
201			
Power Bank	1799	35	1
202			
Wireless Charger	1299	28	1
PRODUCT_ID			
PRODUCT_NAME			
PRICE	STOCK	CATEGORY_ID	
203			
USB-C Cable	499	60	1
204			
Running Shoes			
PRODUCT_ID			
PRODUCT_NAME			
PRICE	STOCK	CATEGORY_ID	
2899	22	2	
205			
Casual Sandals			

9. CRUD OPERATIONS

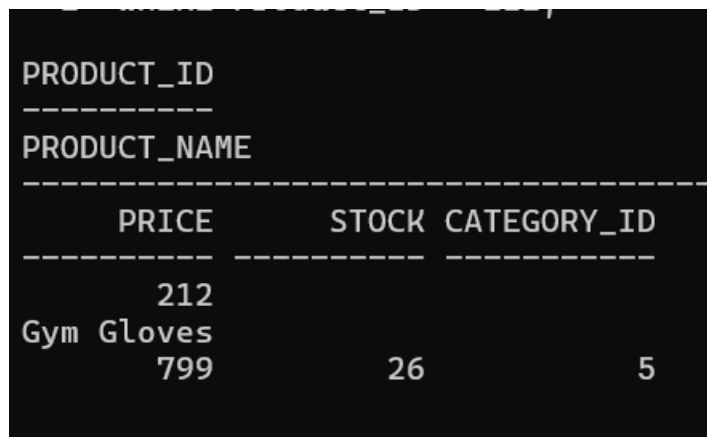
9.1 INSERT RECORD

A new product called Gym Gloves is inserted.

```
INSERT INTO Product
VALUES (212, 'Gym Gloves', 749, 26, 5);

COMMIT;
```

Screenshot



The screenshot shows a terminal window with a SQL query result. The result is a table with five columns: PRODUCT_ID, PRODUCT_NAME, PRICE, STOCK, and CATEGORY_ID. The data row shows that a product with ID 212, named 'Gym Gloves', has been inserted with a price of 799, a stock of 26, and a category ID of 5.

PRODUCT_ID	PRODUCT_NAME	PRICE	STOCK	CATEGORY_ID
212	Gym Gloves	799	26	5

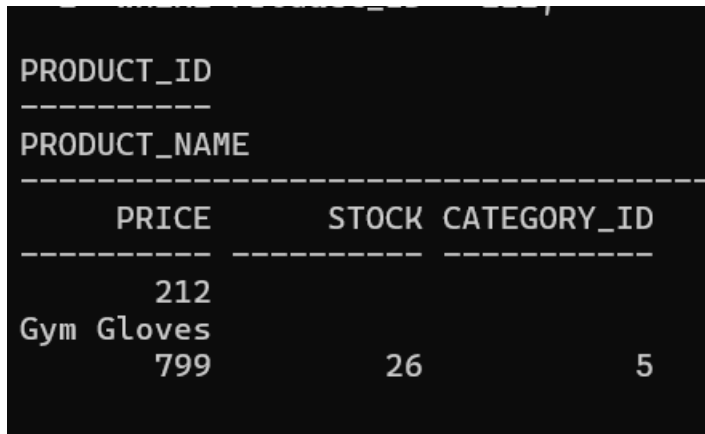
9.2 UPDATE PRICE

The price of Gym Gloves is updated from 749 to 799.

```
UPDATE Product
SET Price = 799
WHERE Product_ID = 212;

COMMIT;
```


Screenshot



A screenshot of a SQL query result displayed on a black background with white text. The result shows a single row for a product. The columns are PRODUCT_ID, PRODUCT_NAME, PRICE, STOCK, and CATEGORY_ID. The values for the row are 212, Gym Gloves, 799, 26, and 5 respectively. The columns are separated by dashed lines.

PRODUCT_ID	PRODUCT_NAME	PRICE	STOCK	CATEGORY_ID
212	Gym Gloves	799	26	5

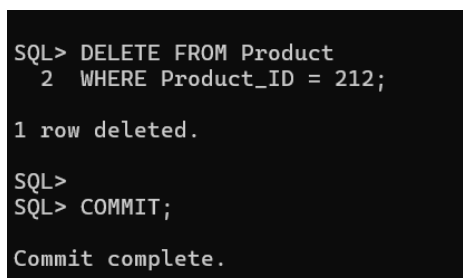
9.3 DELETE A PRODUCT

The Gym Gloves product is deleted from the Product table.

```
DELETE FROM Product
WHERE Product_ID = 212;
```

```
COMMIT;
```

Screenshot



A screenshot of a SQL command prompt showing the execution of a DELETE statement. The prompt shows the command being entered, the result '1 row deleted.', and the final 'Commit complete.' message.

```
SQL> DELETE FROM Product
      2  WHERE Product_ID = 212;

1 row deleted.

SQL>
SQL> COMMIT;

Commit complete.
```

10. CATEGORY-WISE PRODUCT REPORTS

10.1 Display All Products Under Each Category

SQL Query

```
SELECT c.Category_Name,  
       p.Product_Name,  
       p.Price,  
       p.Stock  
FROM Category c  
JOIN Product p  
ON c.Category_ID = p.Category_ID  
ORDER BY c.Category_Name;
```

Screenshot



The screenshot displays the output of an SQL query in a terminal window. It shows three separate result sets, each with a header section and a data section. The first result set lists products under 'Fitness Equipment', the second lists products under 'Footwear', and the third lists products under 'Kitchen Tools'. Each product entry includes its name, price, and stock quantity.

CATEGORY_NAME	PRODUCT_NAME	PRICE	STOCK
Fitness Equipment	Yoga Mat	899	24
Fitness Equipment	Resistance Bands	599	40

CATEGORY_NAME	PRODUCT_NAME	PRICE	STOCK
Footwear	Casual Sandals	999	31
Footwear	Running Shoes		

CATEGORY_NAME	PRODUCT_NAME	PRICE	STOCK
Kitchen Tools	Vegetable Chopper	2899	22

Kitchen Tools		
CATEGORY_NAME		

PRODUCT_NAME		

PRICE		STOCK
-----		-----
Electric Kettle		
2199		16
Mobile Accessories		
Wireless Charger		
1299		28
CATEGORY_NAME		

PRODUCT_NAME		

PRICE		STOCK
-----		-----
Mobile Accessories		
Power Bank		
1799		35
Mobile Accessories		
USB-C Cable		
499		60
CATEGORY_NAME		

PRODUCT_NAME		

PRICE		STOCK
-----		-----
Stationery		
Gel Pen Pack		
199		90

10.2 Count Number of Products in Each Category

SQL Query

```
SELECT c.Category_Name,
       COUNT(p.Product_ID) AS Product_Count
```

```
FROM Category c
LEFT JOIN Product p
ON c.Category_ID = p.Category_ID
GROUP BY c.Category_Name;
```

Screenshot - Output

```
6 GROUP BY c.Category_Name;
```

CATEGORY_NAME	PRODUCT_COUNT
Footwear	2
Stationery	2
Fitness Equipment	2
Kitchen Tools	2
Mobile Accessories	3

10.3 Find Highest-Priced Product in Each Category

SQL Query

```
SELECT c.Category_Name,
       MAX(p.Price) AS Highest_Price
FROM Category c
JOIN Product p
ON c.Category_ID = p.Category_ID
GROUP BY c.Category_Name;
```

Screenshot - Output

CATEGORY_NAME	HIGHEST_PRICE
Footwear	2899
Stationery	349
Fitness Equipment	899
Kitchen Tools	2199
Mobile Accessories	1799

10.4 Display Categories Having More Than 5 Products

SQL Query

```
SELECT c.Category_Name,  
       COUNT(p.Product_ID) AS Product_Count  
FROM Category c  
JOIN Product p  
ON c.Category_ID = p.Category_ID  
GROUP BY c.Category_Name  
HAVING COUNT(p.Product_ID) > 5;
```

Screenshot - Output

```
SQL> SELECT c.Category_Name,  
2         AVG(p.Price) AS Average_Price  
3 FROM Category c  
4 JOIN Product p  
5 ON c.Category_ID = p.Category_ID  
6 GROUP BY c.Category_Name;
```

CATEGORY_NAME	AVERAGE_PRICE
Footwear	1949
Stationery	274
Fitness Equipment	749
Kitchen Tools	1449
Mobile Accessories	1199

10.5 Find Average Product Price Category-Wise

SQL Query

```
SELECT c.Category_Name,  
       AVG(p.Price) AS Average_Price  
FROM Category c  
JOIN Product p  
ON c.Category_ID = p.Category_ID  
GROUP BY c.Category_Name;
```

```
SQL> SELECT c.Category_Name,  
2         AVG(p.Price) AS Average_Price  
3 FROM Category c  
4 JOIN Product p  
5 ON c.Category_ID = p.Category_ID  
6 GROUP BY c.Category_Name;
```

CATEGORY_NAME	AVERAGE_PRICE
Footwear	1949
Stationery	274
Fitness Equipment	749
Kitchen Tools	1449
Mobile Accessories	1199

```
SQL> |
```

11. CONCLUSION

The Product and Category Management System was successfully implemented using Oracle SQL.

The project demonstrates the creation of related tables, implementation of primary and foreign keys, application of constraints, insertion and modification of records, deletion of records, and generation of category-wise reports.

Thus, the project provides a basic understanding of database design, relationships, constraints, CRUD operations, and SQL queries.

12. RESULT

The **Product and Category Management System** was successfully created and executed using Oracle SQL. The required tables, relationships, constraints, CRUD operations, and category-wise reports were successfully implemented.